



Sparser MoE Explorations

Router, communication, and systems
co-design at 2,304 experts

Harry Hanyu

Internship final presentation | July 2026



**Why sparser MoE
changes
the performance
bottleneck**



Roadmap

Context and target workload

- 1 Context and target workload
- 2 Fused-router optimization
- 3 HybridEP sparse exchange
- 4 System integration and model results

Open MoEs are getting wider and more selective

One visible direction is more capacity with a smaller active fraction

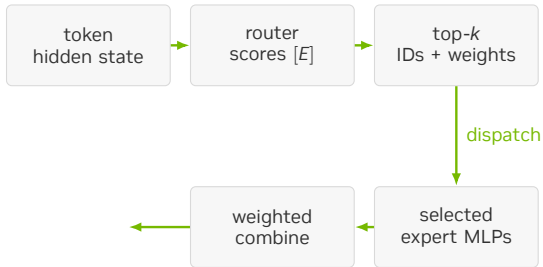
Open model	Scale	Routed experts	Selected/token
Mixtral 8×7B	Early open sparse MoE	8	2
DeepSeek-V3	671B total / 37B active	256	8
Kimi K2	1T total / 32B active	384	8
Qwen3-Next 80B-A3B	80B total / 3B active	512	10

Why this matters for systems

Qwen3-Next also narrows each routed expert to a 512-wide MoE intermediate. Expert GEMMs become cheaper while router rows, route metadata, and ownership exchange remain wide.

MoE is a sparse conditional-compute layer

Only selected experts execute, but every token still routes



```
scores = router(x)           # E scores
ids, weights = topk(scores, K)

y = 0
for j in range(K):
    y += weights[j] * expert[ids[j]](x)
```

The hidden systems cost

Sparse compute reduces MLP FLOPs, not the need to select owners, exchange tokens, preserve weights, and reverse the path in backward.

Latent experts flip the bottleneck budget

Router and communication work shrink more slowly than expert compute

Conventional wider experts

$$\text{expert work} \propto T \times K \times H \times H_{\text{ffn}}$$

Large expert GEMMs amortize routing, launch, and metadata overhead.

Latent / narrow experts

$$H_{\text{ffn}} \downarrow \Rightarrow \text{expert time} \downarrow$$

Router remains $T \times E$; ownership and token exchange remain $T \times K$.

Performance thesis

The right unit of optimization is the entire sparse path: selection, compact metadata, dispatch preprocessing, communication, expert compute, and combine.

The stress model keeps Qwen3-Next's latent experts and widens routing

A 2,304-expert test exposes costs hidden by the public 512-expert shape

Dimension	Public 80B	2,304E test
Routed experts	512	2,304
Selected experts	10	36
MoE intermediate	512	512
Expert parallelism	–	72
Local experts/rank	–	32
Layers	48	24

4.5×

more router
scores

512 → 2,304
experts

3.6×

more selected
routes

10 → 36

Main performance configuration

TP1 / PP1 / EP72, MBS 3, GBS
864, MXFP8, 1F1B,
full-iteration CUDA graph.

The first timeline said data movement had overtaken expert compute

A historical snapshot made the new critical path visible

Region	Time
Attention	3.1 ms
HybridEP dispatch	9.7 ms
Expert MLP	1.1 ms
HybridEP combine	2.5 ms

Three root causes

1. CPU launch overhead from many small kernels.
2. Fused-router degradation at large E and K .
3. Communication and preprocessing overhead.

Remove launch overhead first; then profile the replay-stable GPU path.

First remove CPU launch overhead from the measurement

Make one iteration replayable, then optimize the kernels that remain

```
# capture once
for microbatch in iteration:
    route()
    dispatch()
    grouped_mlp()
    combine()

# replay without Python launch gaps
cuda_graph.replay()
```

- Increase microbatch size where memory permits.
- Capture the full iteration, not isolated kernels.
- Use fused grouped MLP and related MoE kernels.
- Fuse metadata and dispatcher preprocessing kernels.

**1. Fix selection: from
repeated scans to a
router kernel family**



Roadmap

Fused-router optimization

- 1 Context and target workload
- 2 Fused-router optimization**
- 3 HybridEP sparse exchange
- 4 System integration and model results

The original top- k path repeatedly rescanned a 2,304-wide row

Radix selection changes the algorithm from K -dependent scans to fixed passes

Naive selection

```
for slot in range(K):  
    best = -inf  
    for expert in range(E):  
        if expert not already_selected:  
            best = max(best, score[expert])
```

$\mathcal{O}(E \times K)$ -like repeated work

45.36 ms

pre-radix forward

T=65,536; E/K=2,304/36

Radix selection

```
prefix = 0  
for nibble in reversed(range(8)):  
    hist[16] = count_matching(scores, prefix)  
    bucket = bucket_containing_kth(hist)  
    prefix = extend(prefix, bucket)  
gather_values_at_or_above(prefix)
```

8 fixed FP32 radix passes + gather

3.59 ms

after PR #2821

12.6 $\times$ faster

1 warp

owns one token row

deterministic tie handling

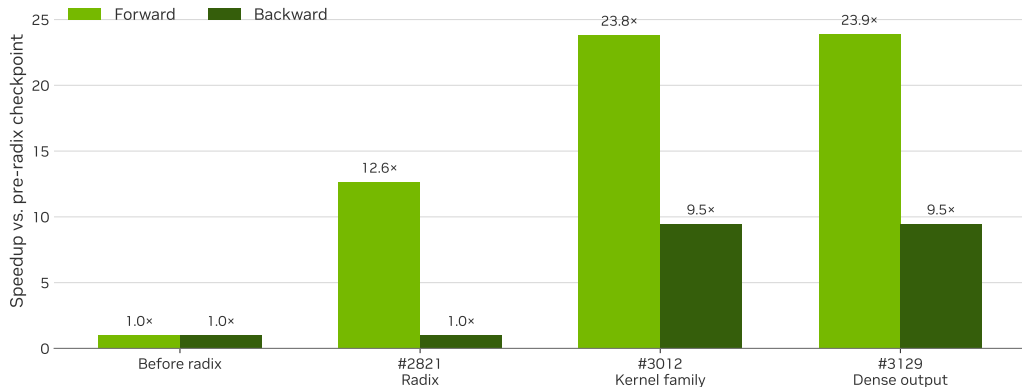
Matched B300 trace-back benchmark; TE PR #2821 Sparser MoE

optimization



Three router stages solved different scaling failures

Matched B300 trace-back: 65,536 tokens, 2,304 experts, top- $k = 36$, softmax

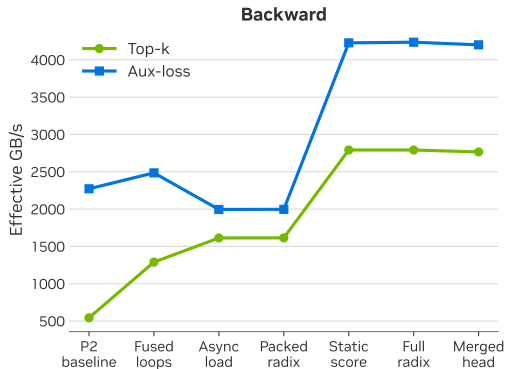
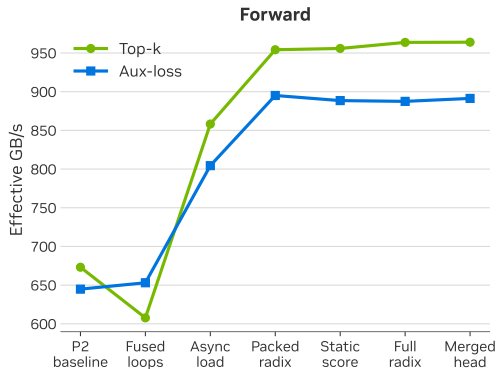


data/trace-back-comparisons; TE PRs #2821, #3012, #3129 Sparser MoE optimization



P3R extended the gain into forward and backward

Incremental 8,192-token result for the target 2,304/36 shape



The optimized router overlaps loads and removes backward scratch

The win is a coordinated kernel design, not a single launch knob

```
prefetch(tile[0], buffer[0])
for tile in token_tiles:
    wait(buffer[current])
    prefetch(tile[next], buffer[next])

    score(buffer[current])
    radix_topk()
    write_selected()
    swap(current, next)
```

- Persistent grid keeps SMs occupied across token rounds.
- Double-buffered `cp.async` overlaps score loads with selection.
- Packed histograms cut radix counters from 32 registers to 4; static score templates remove hot-path branches.
- Two-pass backward removes about 36 KiB/block scratch at $E = 2,304$.

B300 roofs show why the router is not a pure bandwidth kernel

Two deliberately optimistic floors bracket the measured 254.5 μ s result

HBM traffic floor

Semantic traffic is approximately 245 MB:

$$245 \text{ MB} / 8 \text{ TB/s} = 30.7 \mu\text{s}.$$

The measured kernel reaches about 12% of raw HBM bandwidth.

This counts tensor bytes, not cache-line or instruction overhead.

CUDA-core issue floor

Eight radix passes touch 151M scores:

$$8 \times 8192 \times 2304 = 151\text{M}.$$

One FP32 op/visit at 75 TFLOP/s/GPU gives an impossible $\sim 2.0 \mu\text{s}$ floor.

Real visits include masks, shifts, histograms, shuffles, scans, and dependencies.

Compact dense routing removes a $32\times$ metadata expansion

Represent selected IDs directly instead of materializing a bool for every expert

Same choices, smaller state

Bool $[T, E]$	2,304 B/token
int16 $[T, K]$	72 B/token

$$\frac{E}{2K} = \frac{2304}{72} = 32\times.$$

Backward also traverses only the K selected IDs; it no longer asks all E experts to test a K -entry membership list.

```
route_ids = int16[T, K]
probs = fused_router(logits,
                     dense_route_out=route_ids)
hybrid_ep.dispatch(
    tokens, dense_route=route_ids, probs=probs
)
```

One compact producer-to-consumer contract connects router, scan, dispatch, and backward without reconstructing the bool map.

2. Exchange only useful sparse state



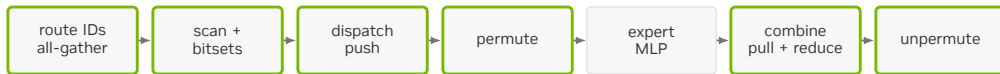
Roadmap

HybridEP sparse exchange

- 1 Context and target workload
- 2 Fused-router optimization
- 3 HybridEP sparse exchange**
- 4 System integration and model results

HybridEP is a pipeline of metadata, remote writes, and remote reads

Each stage has a different sparsity and latency failure mode



Metadata

Avoid bool $[T, E]$ and repeated membership work.

Sparse local work

Skip inactive expert slots in permute and unpermute.

Pipeline

Tune pull depth, grouping, and reduction batch together.

Probability optimization means slice transport, not delete weights

Selected probabilities remain required for weighted activation and gradients

```
# keep mathematical state
ids, weights = router(logits)

# communicate only what this owner needs
for route in selected_routes:
    send(token[route], weights[route])

# backward reuses the selected weights
grad_logits = router_backward(ids, weights, grad)
```

103.0 μs

dispatch with probs

controlled B300 NVL8

249.4 μs

combine with probs

pull + reduce

94.4 μs

without prob payload

+9%

210.0 μs

without prob payload

+19%

The “without probability” rows bound transport opportunity; training still preserves selected weights and their backward dependencies.

Ballot iteration visits active experts instead of all 32 slots

The target has about 4.5 active local experts per token, so 86% of slots are empty

```
bool active = row_id_map[token * E_local + lane] > 0;
uint32_t mask = __ballot_sync(FULL_MASK, active);

while (mask) {
    int expert = __ffs(mask) - 1;
    mask &= mask - 1;
    int peer = __shfl_sync(FULL_MASK, my_peer, expert);
    move_selected_token(peer);
}
```

4.27×

permute speedup
396.1 → 92.7 μ s

2.04×

unpermute
speedup
269.5 → 131.8 μ s

Launch policy

Use 32 threads/token at latent $H = 512$; larger hidden sizes keep wider thread groups to expose enough memory-level parallelism.

Dense int16 routes enable one-pass scan bitsets

Build expert and rank membership once, then use cheap bit tests downstream

```
local_mask = 0
rank_mask = 0

for slot in range(K):
    expert = dense_route[token, slot]
    local_mask |= 1 << (expert % E_local)
    rank_mask |= 1 << owner_rank(expert)

if local_mask & (1 << expert):
    process_active_expert()
```

Target isolated scan

Best 512-thread template:

525.6 $\rightarrow$ 331.7 μ S (1.58 $\times$)

No permute-fusion metadata in this isolated comparison.

The scan consumes the router's compact $[T, K]$ output directly; no bool route-map reconstruction is required.

Combine tuning is a constrained pipeline search

The 2,304-expert production tuple is a measured contribution, not a default guess

```
for group in [1, 2, 4, 8]:
    check_fifo_slots_per_pipeline()
    for g2s_depth in candidates:
        for reduce_batch in candidates:
            time_combine()
            time_combine_plus_unpermute()
            verify_correctness()

choose shallowest queue hiding pull latency
validate again in the full EP72 launch
```

Parameter	NVL8 study	2304E / EP72
Combine SMs	32	32
G2S stages	64	72
S2G stages	8	8
Tokens/group	2	2
Reduce batch	16	72

The EP72 tuple matches the wider

communication domain; neither tuple is universal.

The winning combine tuple balances lookahead and SM efficiency

Group 2 uses both pipelines while preserving enough FIFO depth



Enough lookahead

32 FIFO slots / ~ 8 source ranks ≈ 4 tokens prefetched.

No excess depth

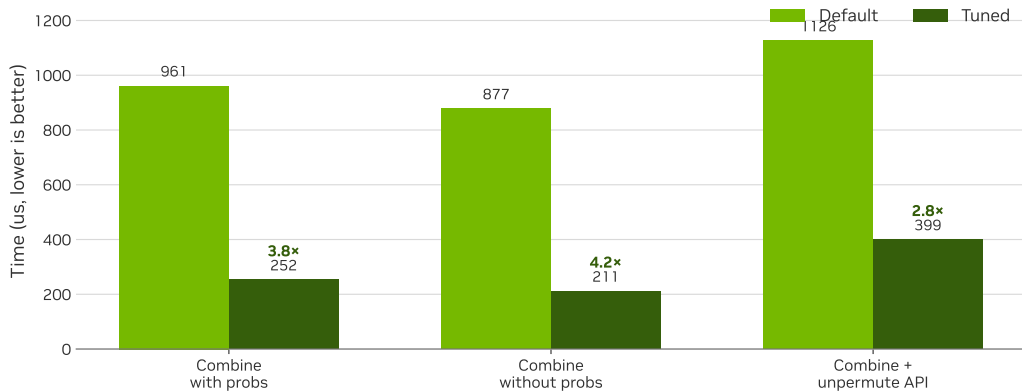
G2S 64 $\rightarrow$ 128 changed 246.1 to 245.0 μs : saturation.

Use fewer SMs

Group 2 matched group 1 throughput using 32 instead of 64 SMs.

Coordinated combine tuning cut the slow path by 2.8--4.2×

Controlled H=512, local-E=32, top-k = 36 comparison



notes/hybrid-ep-sparse-opt-new.md; controlled B200 comparison Sparsen
MoE optimization



Combine remains structurally harder than dispatch

Push can continue; pull must wait, synchronize, reduce, and then store

Dispatch: push / S2G

- 116.8 μ s
- 650 GB/s
- 58% long-scoreboard stall
- 0.05% barrier stall

Combine: pull / G2S + reduce

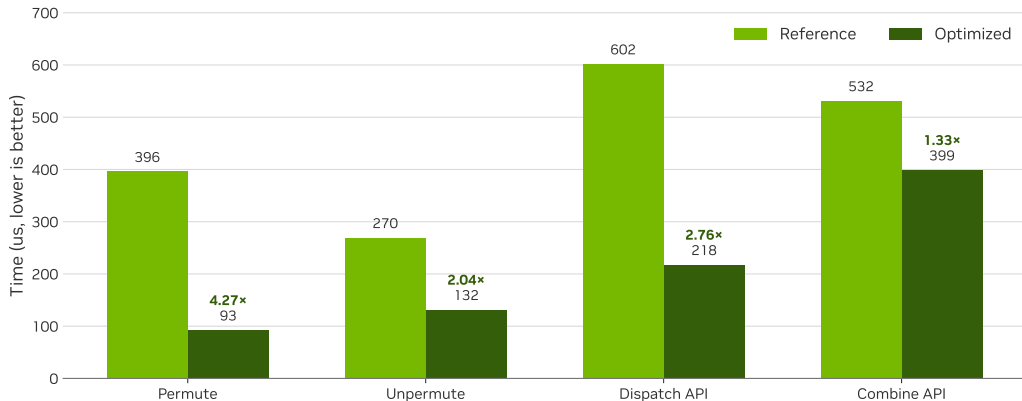
- 254.4 μ s
- 271 GB/s
- 27% long-scoreboard + 26% wait
- 15% barrier stall

Design consequence

Fusing combine with unpermute serialized the consumer behind slowly produced chunks: 434 μ s fused versus 381 μ s standalone, about 14% slower.

The controlled 8-GPU microbenchmark validates the sparse path

B300 NVL8, H=512, 8,192 tokens/rank, local-E=32, top- k = 36



notes/hybrid-ep-sparse-opt-new.md; tuned reference vs tuned sparse path
Sparsers MoE optimization

A custom metadata all-gather was positive, but it is not the main result

Use direct registered-buffer writes only where the collective is actually on the path

~15%

historical isolated
gain

custom all-gather vs
NCCL

32×

logical metadata
reduction

bool $[T, E]$ to int16 $[T, K]$

- Writes directly into HybridEP registered buffers.
- Avoids a separate NCCL-buffer-to-PyTorch-tensor copy.
- Benefit is collective- and topology-specific.
- Compact representation reduces bytes before the collective implementation matters.

Scope: an isolated communication optimization; the full-model gain is not attributed to all-gather alone.

Profiling rejected three attractive but wrong shortcuts

Optimization also means narrowing the design space

Direct permute

Repeated remote writes and address work moved the bottleneck into dispatch; the saved 92 μ s permute did not compensate.

Fused combine

Producer-consumer serialization left the best fused path 14% slower than standalone.

Warp-pruned scan

Extra bitset initialization and coordination cost more than the skipped ballots on key shapes.

**3. Integrate once;
measure the whole
model**



Roadmap

System integration and model results

- 1 Context and target workload
- 2 Fused-router optimization
- 3 HybridEP sparse exchange
- 4 System integration and model results

Megatron Core keeps the full sparse path replayable

Authorship boundaries are explicit

Colleagues' system contributions

- Paged stash for bounded activation storage.
- Full-iteration CUDA graph capture and replay.
- 1F1B MoE communication/compute overlap.
- Capture-safe memory release without deferred-free inflation.

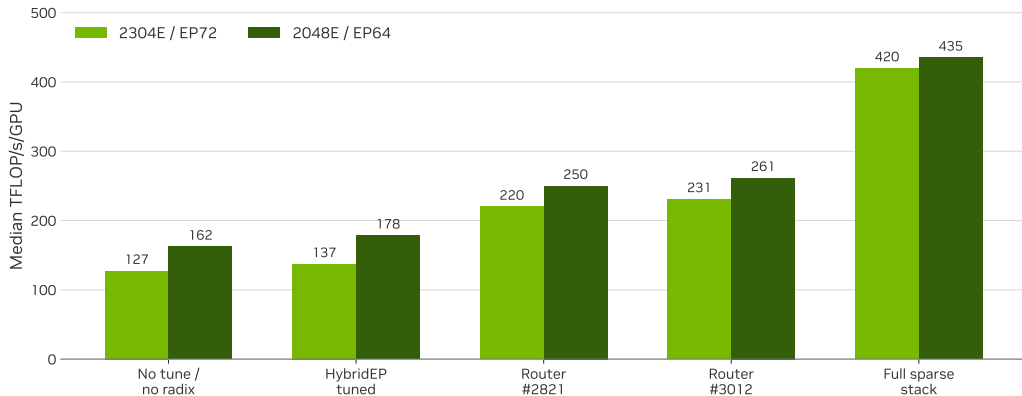
This work's integration surface

```
if te.has_dense_route_output() and \
    dispatcher.accepts_dense_route():
    route = int16[T, K]
    fused_router(..., route_out=route)
    hybrid_ep(..., dense_route=route)
else:
    compatibility_bool_route()
```

Capability detection preserves backward compatibility.

The full model shows where each stage moved the critical path

OCI-AGA GB300, 1F1B + paged stash + full-iteration graph; median after warmup 5



qwen3_next_2606_image_sweep_20260728; wgrad-fix runs, 2026-07-30
Sparsers MoE optimization

The full sparse stack reaches 420--435 TFLOP/s/GPU

The final jump is cross-layer: router, compact routes, sparse preprocessing, bitsets, and tuned combine

419.7

2304E / EP72 median

TFLOP/s/GPU

127.2 no-tune baseline; 3.30×

435.1

2048E / EP64 median

TFLOP/s/GPU

162.2 no-tune baseline; 2.68×

530 / 423

iterations observed

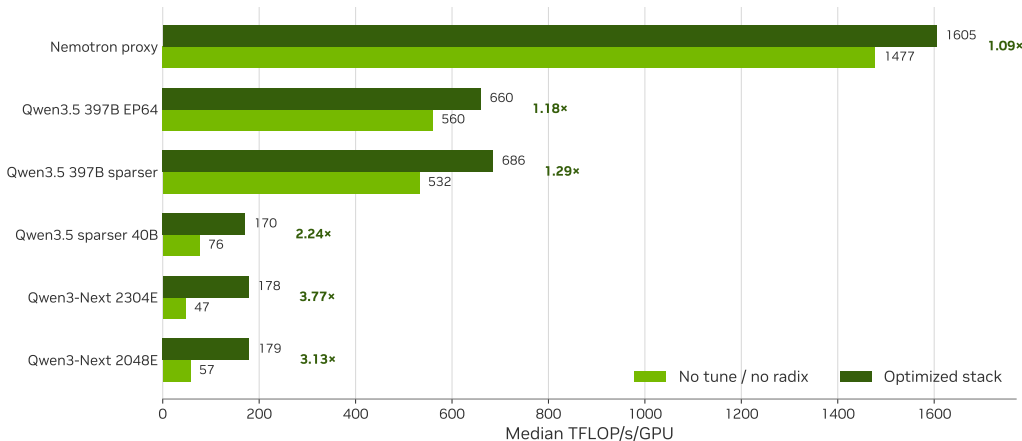
2304E / 2048E full-stack runs

Matched stack

Both shapes use the same 26.06 image family, mock data, MBS 3, 24 layers, full 1F1B, paged stash, full-iteration graph, and the wgrad fix that disables the CuTe DSL fused GroupedMLP wgrad subpath. Isolated kernel speedups are not added together.

Routing stress amplifies the MoE-only gain

OCI-AGA GB300; 155 post-warmup iterations; matched full graph + paged stash



moe_perf_canonical_matrix_20260730; median TFLOP/s/GPU Sparser MoE optimization

The largest MoE-only gains are the widest sparse shapes

The optimized stack is useful beyond one 2,304-expert model

3.77×

Qwen3-Next 2304E

47.2 → 177.9

3.13×

Qwen3-Next 2048E

57.4 → 179.4

2.24×

Qwen3.5 sparser 40B

76.1 → 170.3

Boundary case

Nemotron's proxy shape improved only 1.09×: when expert compute is already large, router and sparse communication are a smaller fraction of the module. This matrix is MoE-only, without 1F1B; full graph and paged stash are matched.

What the evidence supports

Separate isolated mechanism gains from system throughput

Supported claims

- Radix fixes pathological large- E , large- K selection.
- Compact int16 routes reduce logical metadata 32 $\times$.
- Ballot skips inactive local experts.
- Combine needs coordinated queue/group/batch tuning.
- Gains generalize most strongly to sparse, wide expert shapes.

Claims to avoid

- Do not add isolated speedups to predict training speed.
- Do not call the EP72 tuple a universal default.
- Do not say probabilities are removed from training.
- Do not attribute colleagues' graph/stash/1F1B work to this project.
- Do not mix MoE-only and full-model scopes.

Sparser MoE became practical through one compact invariant

Selected routes stay compact from the router to the owning expert and back

Select

TE fused router

Represent

int16 $[T, K]$

Exchange

HybridEP

Pipeline

Combine

Replay

Megatron Core



Radix + async per-sistent kernels

23.9×

router forward

pre-radix → dense output

No bool $[T, E]$ expansion

1.84×

HybridEP API total

1,133.7 → 617.1 μ s

Probability slicing + bitsets

Tune stages, groups, and batch

419.7

full-model TFLOP/s/GPU

2,304 experts / EP72

Full-stack, capture-safe execution

Appendix



Roadmap

Appendix

5 Appendix

Combine sensitivity explains the final group-2 choice

More stages or groups are not monotonically better; EP72 validates 32 SMs / 72 / 8 / group 2 / batch 72

Configuration	Time	Output BW	Interpretation
Group 1, 64 SMs	252 μ s	265 GB/s	Full FIFO depth; half the warp resources underused
Group 2, 32 SMs	254 μ s	263 GB/s	Same throughput with half the SMs
Group 4, 32 SMs	502 μ s	133 GB/s	FIFO too shallow to hide pull latency
G2S 64 $\rightarrow$ 128	246 $\rightarrow$ 245 μ s	flat	Queue depth already saturated

Exact matched router times for the target shape

65,536 tokens, 2,304 experts, top- k = 36, softmax

Checkpoint	Forward	Backward	Output
Before large top- k support	45.358 ms	3.552 ms	sparse bool
PR #2821	3.587 ms	3.553 ms	sparse bool
PR #3012	1.904 ms	0.375 ms	sparse bool
PR #3129	1.899 ms	0.375 ms	dense int16

Reading the table

PR #2821 changes the selection algorithm; PR #3012 redesigns forward and backward; PR #3129 changes the API representation without regressing the target forward path.

Primary evidence map

The deck is reproducible from local data and official model sources

Topic	Primary source
Persistent content memory	<code>../sparser_moe_final_presentation_content.md</code>
Router trace-back	<code>data/router_trace_back.csv</code> ; <code>data/router_fix_p3R_*.csv</code>
HybridEP	<code>hybrid-ep-sparse-opt-new.md</code> ; <code>hybrid_ep_rebased_bw_results.md</code>
Dense scan	<code>isolated-scan-bench.md</code>
Full-model sweep	<code>qwen3_next_2606_image_sweep_20260728/README.md</code>
MoE-only matrix	<code>moe_perf_canonical_matrix_20260730/README.md</code>
Open models	Mixtral; DeepSeek-V3; Kimi K2; Qwen3-Next
Hardware roofs	NVIDIA HGX B300; NVIDIA HGX AI Factory components

Thank you

